

Another change from Assignment 1 was improving console feedback for ongoing effects. `PoisonStatus`,

BurnStatus, and Fire now print remaining turns or effect feedback. This helps the player understand how long ongoing effects will last and directly addresses the previous issue where turn-based effects were not clearly communicated.

2. Justification for the New REQ1 Economy Requirement

For REQ1, I implemented the economy system through the Supercomputer, which allows workers to buy and sell items. The main new classes and interfaces are Wallet, WalletHolder, Supercomputer, TransactionAction, BuyItemAction, SellItemAction, Purchasable, Sellable, FireSpawner, FireHazard, BurnStatus, and AccessLevel.

The Wallet class manages credits and enforces the maximum capacity of 1,000 credits. It also prevents credits from becoming negative. This keeps credit-related rules inside one cohesive class instead of duplicating wallet validation logic across multiple actions. ContractedWorker implements WalletHolder, allowing transaction actions to access the worker's wallet through an abstraction rather than depending directly on the concrete ContractedWorker class.

BuyItemAction and SellItemAction both extend TransactionAction. The purpose of TransactionAction is to centralise shared wallet access logic, which supports DRY and avoids duplicated transaction code. BuyItemAction depends on the Purchasable interface, while SellItemAction depends on the Sellable interface. This means the actions do not need to know whether the item is an Apple, Lantern, AccessCard, or another concrete item. Instead, each item defines its own price and transaction effect through the relevant interface.

This design improves cohesion because item-specific rules are placed inside the item classes that own those behaviours. For example, Apple handles its own poison effect when sold, Cookies calculates its own dynamic selling price, Lantern handles its burn and fire-spawning effects, and AccessCard handles its level-specific purchase effects. This also supports the Open-Closed Principle because new sellable or purchasable items can be added by implementing Sellable or Purchasable, without changing the core transaction actions.

3. Alternative Designs and Trade-offs

Alternative Design 1: Put all transaction logic inside Supercomputer

Pros:

This would make the initial implementation simpler because all buying and selling logic would be located in one class.

Cons:

However, this would make Supercomputer responsible for too many item-specific rules, such as Apple poisoning, Lantern fire spawning, CRT Monitor effects, and Access Card hidden fees. This would reduce

cohesion, increase coupling, and likely create long conditional logic.

Decision:

I rejected this design and used Sellable and Purchasable instead. This allows each item to manage its own transaction behaviour, making the system easier to extend and maintain.

Alternative Design 2: Pass the purchasable stock list from EclipseNebula into Supercomputer

Pros:

This would reduce coupling because Supercomputer would only receive a list of Purchasable objects and would not need to know concrete purchasable item classes.

Cons:

However, this would move shop configuration into EclipseNebula, making world setup longer and less focused. Since Supercomputer is described as the trading terminal, keeping its catalogue inside Supercomputer makes the shop configuration simpler, centralised, and easier to understand.

Decision:

I chose to let Supercomputer initialise its own catalogue. The trade-off is that Supercomputer still depends on concrete purchasable item classes, but item-specific purchase effects are still handled through the Purchasable interface.

4. Alignment with Design Principles

This design aims to improve low coupling, high cohesion, DRY, and SOLID principles. Coupling is reduced because transaction actions depend on interfaces such as Purchasable, Sellable, and WalletHolder, rather than directly depending on concrete item or actor classes. Cohesion is improved because Wallet manages credits, Supercomputer provides trading actions, TransactionAction handles shared transaction behaviour, and item classes manage their own transaction effects.

The design also avoids the Assignment 1 issue of relying heavily on instanceof. Instead, the code asks whether an object has a required capability, such as being sellable, purchasable, drinkable, or a fire hazard. The UML class diagram also reflects the important relationships between actions, interfaces, and item classes, helping the design stay aligned with the implementation. Overall, this approach is more maintainable and extensible than a design based on large conditional statements inside one central class.